Technical Explanation Report

Distributed and Streaming-based Client-Server Collaboration Application

Implementation: Collab LAN | Evidence recorded: 2026-10-17T14:10:17+00:00

1. Objective and system architecture

Collab LAN provides nickname login, live user discovery, private/broadcast chat, and reliable binary file transfer for a trusted LAN group. It uses Python raw TCP/UDP sockets, application-layer reliability, Fernet encryption, and a plain JavaScript browser interface. Core networking is separated from data processing and presentation.

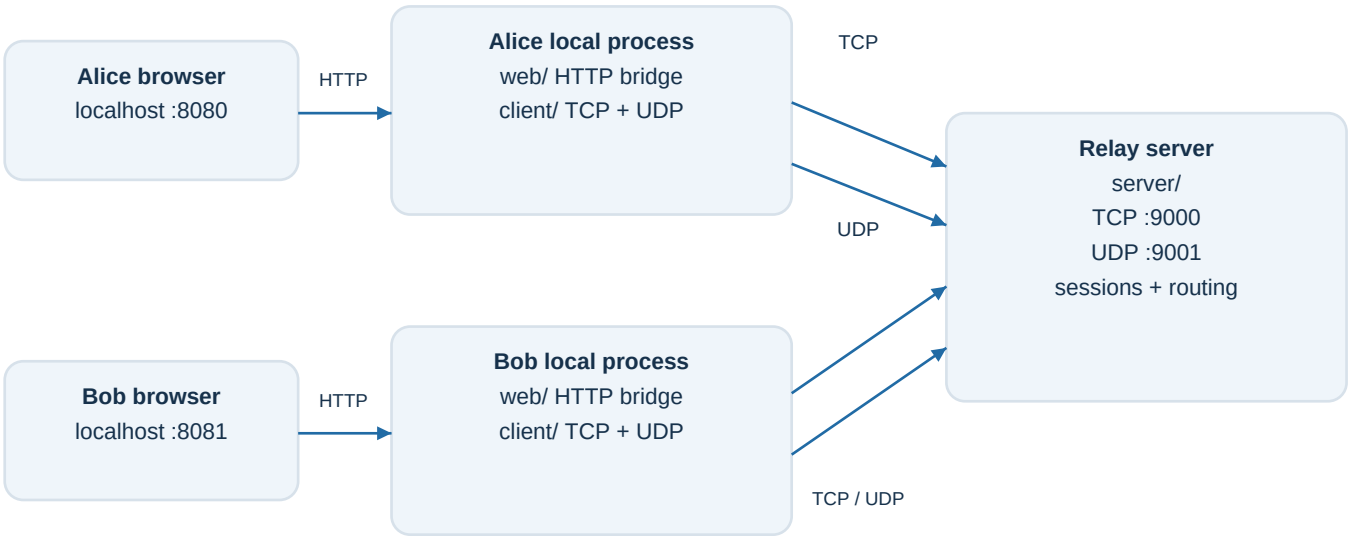


Figure 1. Browsers talk only to their local HTTP bridge. Client processes communicate with the relay over encrypted TCP and UDP. Arrows identify connection paths; both transports exchange traffic in both directions. A CLI can replace the browser/HTTP bridge.

Transport responsibilities

TCP: encrypted login, session control, large presence snapshots, and final chat/file results. **UDP:** encrypted heartbeat, path verification, presence, chat/file payloads, and per-hop ACK/retry. The relay sees application contents; this is not end-to-end encryption between clients.

Verified snapshot: 237 tests completed with OK; includes 20 simultaneous clients. This is functional localhost evidence, not a WAN throughput or congestion-control benchmark.

2. Modular implementation and concurrency

Package / entry point	Responsibility
collab/client/	core.py owns sockets and lifecycle; cli.py owns terminal commands and exit handling; chat_sender.py waits for delivery results; chat_inbox.py handles per-stream ordering; file_transfer.py streams and verifies files.
collab/server/	core.py manages sessions, bounded connections and supervision; routing.py handles chat admission/retries; file_transfers.py tracks fixed recipient sessions and file results.
collab/transport/	tcp.py provides persistent framing/deadlines; udp.py provides size-aware encoding; security.py owns authenticated envelopes/replay state; handshake.py binds login to fresh nonces.
collab/shared/	protocol.py validates business messages; limits.py defines protocol defaults; concurrency.py provides bounded pools; resilience.py handles retry/backoff and independent cleanup.
collab/web/	handlers.py serves JSON and fixed static routes; static/index.html, style.css and app.js contain the frontend; runtime.py bounds work; cli.py starts the interface.
Compatibility modules	common.py re-exports shared APIs and shutdown/control defaults. webui.py retains PAGE and the web command; core.main() lazily forwards to client.cli. Package entry points preserve python -m commands.

Worker ownership

The server has five critical loops: accept, UDP receive, retransmission, session expiry, and presence. One supervisor coordinates fatal-worker shutdown. Each accepted connection has a tracked handler; each logged-in session has a TCP writer with bounded control/chat/file mailboxes. Sockets are registered before handlers start.

A connected client has TCP receive, UDP receive, heartbeat, and file/disk workers. Slow writes and hashing run in the file worker rather than the UDP listener. HTTP adds two send workers plus four waiting slots, two upload slots, and eight handler workers with capacity for 16 admitted connections.

Lock and resource boundaries

Registry locks protect short state changes. Encoding, network sends, disk I/O, and joins run outside the global server lock. UDP session gates are acquired in token order, with session identity checked again before sending. Startup gates prevent workers processing partially initialized sessions. Cleanup uses a shared 3-second deadline and retains unfinished resources for a later retry.

Source: collab/server/core.py; collab/client/core.py; collab/web/runtime.py; collab/shared/concurrency.py.

3. Application protocol specification

Message / transport	Payload and purpose
hello / challenge (TCP)	hello: client_nonce. challenge: client_nonce + server_nonce. Each nonce contains 32 random bytes encoded as 64 hex characters.
login / welcome (TCP)	login: name and both nonces. welcome: token, udp_port, session and both nonces. The public session context differs from the secret authorization token.
error / logout / users (TCP)	error carries a bounded message; logout ends the session. Server users responses carry names[] and nonnegative revision.
chat_status (TCP)	id, state (delivered/failed/rejected), original recipients[], delivered[], failures[{name, reason}].
File control (TCP)	file_status reports accepted/final results. file_receipt confirms recipient saving or failure; file_abort and file_cancel terminate transfer work.
heartbeat / presence (UDP)	Client packets carry from + token. Server presence carries sorted names[] + revision; clients ignore older revisions.
path_challenge / response (UDP)	A nonce proves that a new candidate endpoint can receive and answer an authenticated challenge before registration.
data/chat (UDP)	id, from, to, kind, payload, meta. Forwarded metadata adds sender_session, original channel and per-recipient stream sequence.
data/file_start (UDP)	meta: transfer, filename, size, count, sha256; forwarded packets also include sender_session.
data/file_chunk / end (UDP)	chunk: transfer + index and base64 payload. end: transfer and empty payload. Receiver checks chunk length against declared file size.
ack (UDP)	id identifies a hop-local packet. Client ACKs also carry from + token. Server-to-client payloads do not expose authorization tokens.

Framing and validation

Business payloads are compact UTF-8 JSON objects. TCP carries a 4-byte big-endian ciphertext length followed by a Fernet token. Its JSON payload limit remains 2,000,000 bytes, with a separately computed ciphertext ceiling. UDP carries one Fernet token per datagram, at most 1,100 bytes including encryption overhead; a raw file chunk is 320 bytes.

TCPReader preserves partial headers/bodies across timeout and reads only missing bytes. EOF mid-frame, invalid lengths, authentication failure or malformed JSON terminate that connection. UDP is authenticated and schema-checked before replay state, endpoint, liveness or application state can change. Invalid packets receive no ACK.

Source: collab/shared/protocol.py; collab/transport/tcp.py; collab/transport/udp.py.

4. Security Envelope and anti-replay window

The authenticated plaintext is a 27-byte binary header followed by compact JSON. Fernet encrypts and authenticates both together. The header is **inside** the Fernet token; it is not an unauthenticated routing prefix.

```
wire TCP = uint32_be(token_length) || Fernet(header || JSON)
wire UDP = Fernet(header || JSON)
header format = !B16sBBQ   (network byte order)
```

Version	Context	Direction	Transport	Sequence
1 byte	16 bytes	1 byte	1 byte	8 bytes

Field	Validation / meaning
Version (1 byte)	Must equal 1; no downgrade to legacy plaintext or old UDP JSON framing.
Context (16 bytes)	All-zero context during nonce-bound handshake; a new public session UUID after welcome. It is separate from the authorization token.
Direction (1 byte)	1 = client-to-server; 2 = server-to-client. Reflected frames in the wrong direction are rejected.
Transport (1 byte)	1 = TCP; 2 = UDP. A frame cannot be reused across transports.
Sequence (8 bytes)	Unsigned integer from 1 to 2^64 - 1. Separate send/receive state per channel, direction and session. Exhaustion is an explicit error.

Bounded UDP replay window: 4,096 sequence numbers

The receiver keeps a highest sequence number and a 4,096-bit bitmap. A higher sequence shifts the bitmap, marks the new sequence and discards bits outside the window. A large jump clears the window before marking the newest packet. An older sequence is accepted only if it remains within the window and its bit is unset. Duplicate and out-of-window packets are dropped without ACK or liveness refresh.

```
receive: 3 -> accept; 1 -> accept; 2 -> accept; 1 -> reject
receive: 4099 -> accept; 3 -> reject (distance = 4096)
```

For UDP, schema, session, direction and transport checks precede replay-state commitment. Each actual UDP retry uses a fresh security sequence while retaining the application ID, so application deduplication can ACK a legitimate retry without displaying it twice. TCP instead requires exactly the next sequence number. Counters are reset for every new session.

Trust boundary: one COLLAB_KEY is shared by the group. Key holders can decrypt and forge group traffic; there is no per-user cryptographic identity, TLS or forward secrecy. Replay protection does not detect every valid packet withheld before first delivery. Fernet timestamps are not used as the main freshness check, avoiding a wall-clock synchronization dependency.

5. Login, endpoint validation and presence

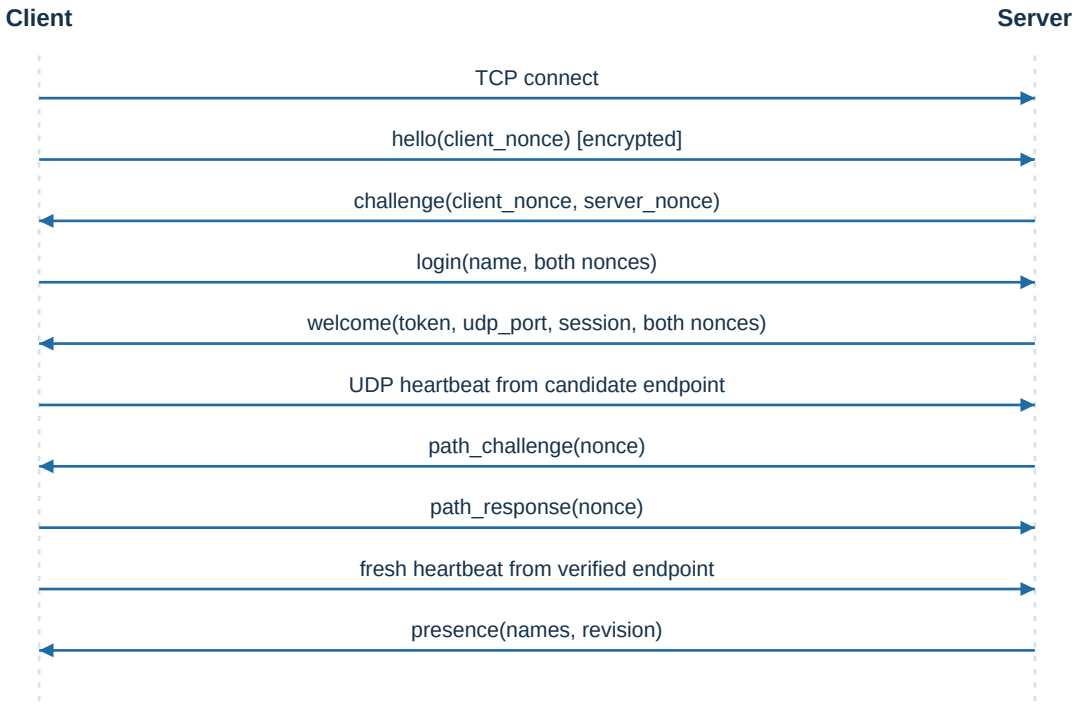


Figure 2. All messages after TCP connect are protected. Fresh nonces bind a login/welcome exchange to this attempt; old login/welcome ciphertext cannot complete a new handshake. After welcome, both ends switch to the new session context and reset channel counters.

Deadlines and initialization

TCP establishment allows 5 seconds; the complete client handshake has 5 seconds, and the server has 4 seconds from accept. These are monotonic total deadlines, so drip-fed bytes do not extend them. Username registration is atomic. Server ready is set only after welcome and writer startup; clients become connected after local UDP setup, first heartbeat send, and worker startup. Connected does not guarantee UDP reachability.

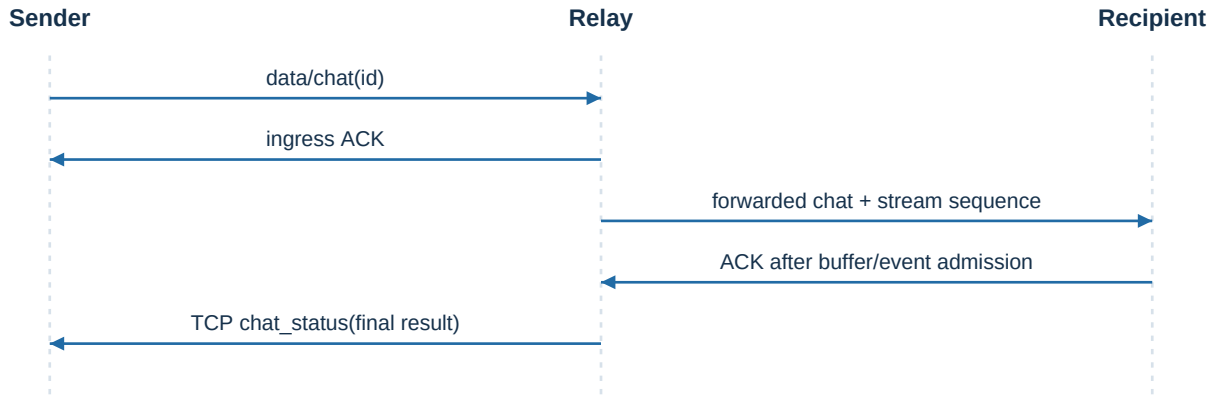
Path verification and session expiry

A new UDP source address does not immediately replace the verified endpoint. One candidate per session holds a random challenge for 2 seconds; new challenges are rate-limited to one per 0.5 seconds. A matching response must come from that candidate before expiry. Path control alone never refreshes liveness; the old endpoint remains usable until replacement succeeds.

Valid business UDP activity refreshes last_seen. After 10 seconds without it, expiry closes TCP and removes the session; checks run every 0.5 seconds. Logout/EOF/errors use the same session-identity-safe cleanup. Revision increases on online membership changes, not endpoint changes. Heartbeats recover presence; snapshots above the UDP ceiling use TCP users.

6. Message and file-transfer lifecycles

Chat: hop ACK versus application acceptance



The relay caches source-session + ID, normalized content and a fixed recipient-session set. Retries cannot add recipients or change content. A recipient ACK follows valid buffer/event admission; it does not imply a human has read the message. Streams buffer up to 64 messages, wait at most 1 second for a gap, then report the gap and advance without blocking other streams. Server delivery deadline: 5 seconds; sender confirmation deadline: 10 seconds.

Files: verified disk saving before final success

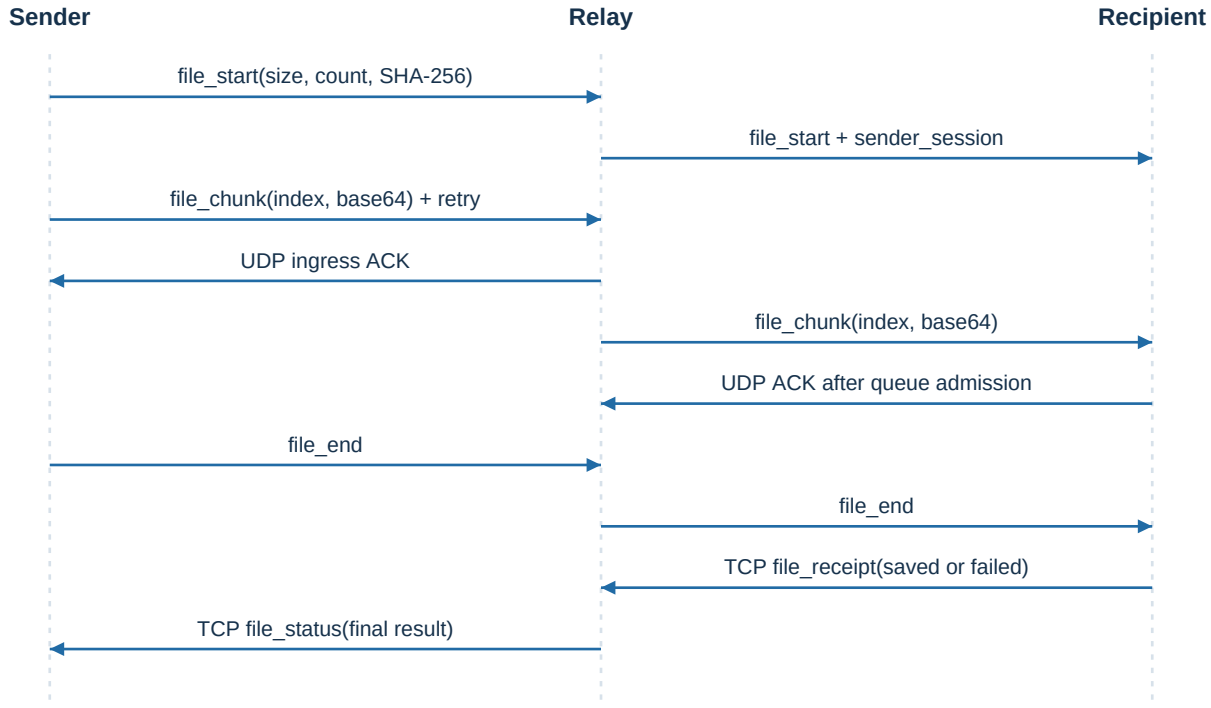


Figure 3. ACKs exist on both UDP hops; the diagram groups repeated chunk/ACK exchanges. Sender hashing and reads are streamed. The recipient's file worker seeks/writes a temporary file by chunk index, verifies SHA-256 by streaming reads, and renames only if the local session still matches. Empty files are supported. file_complete is local receive success; file_sent follows successful final recipient receipts, not relay ingress.

The source waits for all original recipients; partial failures include delivered/failed names. Pending packets retain source/destination session tokens internally and are removed on session end. The UI exposes separate send/receive progress with transfer identity; 100% chunk progress still waits for saving/recipient confirmation.

7. Challenges and solutions

Keeping modular boundaries while preserving commands

The initial single-file client/server layout mixed socket ownership, routing and UI concerns. Packages now separate transport/security, shared validation, client chat/file services, server routing and the HTTP bridge. Compatibility exports and package entry points retain existing imports and commands. Timing defaults are centralized in shared/limits.py. Public interfaces have Python 3.10+ annotations; caller-held-lock and cleanup invariants document the existing model. External CSS/JS preserve UI behavior, and a lazy core.main() wrapper avoids a CLI/core import cycle.

Timeouts without corrupting a byte stream

A TCP timeout may occur after part of a header or body has arrived. Restarting an ad-hoc reader would lose framing. One persistent TCPReader retains those bytes and bounds each recv to missing data. Total monotonic handshake deadlines stop silent/drip clients. Post-login poll timeouts preserve state; malformed frames fail closed instead of attempting stream resynchronization.

Reliability without duplicate delivery or stale-session leakage

Security counters and application IDs serve different purposes. Retries receive a new authenticated counter but keep their application ID. Routing caches pin original recipients and compare content; destination-session checks prevent retries reaching a new login under the same nickname. Chat ACK follows admission, while a file's final result requires verified saving. Bounded per-stream gap waiting avoids one conversation blocking another.

Disk and network work must not block shared workers

The UDP listener retains valid chunks in a bounded queue; the file worker writes/hashes outside the state lock. Queue admission precedes ACK, so rejected work can retry. Per-session TCP writers bound slow-reader effects. Global pending limits reserve chat capacity, and batches prefer 96 chat / 32 file packets out of 128, with unused slots redistributed and rotation within groups.

Authenticated traffic still needs replay and path checks

Encryption alone did not stop a captured heartbeat refreshing an active session or changing its endpoint. The authenticated context/direction/transport fields, bounded replay bitmap and endpoint challenge now make these separate checks explicit. Client UDP also verifies the actual server IP/advertised UDP port. These mechanisms protect against outsiders; possession of the group key still grants group-wide trust.

Cleanup must remain independent under failure

Critical server workers report the first error to a supervisor rather than joining one another. Handler/writer errors close only their own session. Cleanup attempts socket shutdown, queued-work cancellation, temporary-file cleanup and joins even if another step fails, using one deadline. EINTR/EAGAIN retry and interruptible UDP backoff avoid busy loops; incomplete resources stay tracked and block unsafe restart.

8. Test evidence and validation boundaries

Recorded run: 237 tests in 40.956 seconds; **OK**. Environment: Darwin / Python 3.12.14. Recorded UTC: 2026-10-17T14:10:17+00:00.

```
python -m unittest discover -s tests -v
```

Test module	Tests	Evidence covered
chat	31	Fixed recipients, ACK loss, ordering, gaps, limits and delivery results
concurrency	18	20 simultaneous clients, capacity, startup/shutdown and slow peers
files	35	Streaming, SHA-256, empty files, partial failures, retries and cleanup
handshake	21	Partial frames, total deadlines, initialization and rollback
integration	7	Real encrypted sockets, unicast/broadcast, file hash and invalid UDP
protocol	12	Framing, schema, sizes, Unicode and invalid payload rejection
refactoring	8	CLI import/legacy entry, timing aliases, annotations and static loading
resilience	36	Worker failure supervision, CLI errors, subprocess death and deadlines
security	31	Encrypted envelopes, replay, wrong keys, nonce/path checks and rollback
sessions	25	Tokens, expiry, endpoint changes, presence and session isolation
web_concurrency	7	HTTP admission, upload ownership, cancellation and pool cleanup
web_ui	6	HTTP messages/presence/progress, static MIME/routes and JavaScript rendering

What these tests demonstrate

The 20-client test uses barriers to race login, unicast/broadcast and close, comparing exact content and receiver counts. Socket localhost tests combine with fake clocks, Events, Barriers and injected errors for deterministic failure boundaries. The invalid-UDP integration case uses a valid security envelope and observes actual schema rejection, rather than passing only because a legacy envelope was rejected.

Static-route tests verify exact resource bytes, MIME types and allowlisted paths. Import tests cover both CLI/core orders, loading from another working directory and clear failure when resources are missing. The JavaScript test requires Node.js and is explicitly skipped if unavailable; browser HTTP integration remains Python-based. Failure-injection cases intentionally log tracebacks, so the final unittest summary and exit code determine success. Test counts are checked against the current source inventory whenever this report is generated.

The current evidence does not establish Windows/Linux execution, physical-LAN firewall behavior, WAN performance, or broad loss/latency benchmarks. A 20-client functional scenario is not a throughput claim. Detailed output: docs/test_results.txt.

9. Limits, reproduction and submission

Default / boundary	Current behavior
File and receive limits	25 MiB/file; 320-byte chunks; two outgoing and two incoming files per client; 50 MiB total declared incoming size. File write queue: 256 jobs.
Server admission	64 TCP connections including handshake; 8,192 pending packets total; 1,024 per recipient. Limits are configurable keyword-only constructor parameters.
Retry and resource bounds	Client window: 6 packets; client attempts: 15; relay attempts: 12; retry interval: 250 ms. Chat/file result mailboxes and caches have explicit bounds.
File lifetime	Default idle timeout: 30 seconds; total transfer deadline: 15 minutes. Incomplete transfers are cancelled and temporary resources are cleaned.
Remaining scope	No BBR/CUBIC, persistent offline messages, automatic reconnect, resumable transfer, or per-user key isolation. Browser HTTP upload still buffers JSON/base64. UI history is not persistent or synchronized across tabs.

Run the demo

Create/activate a Python 3.10+ virtual environment, install requirements.txt, generate one key with python -m collab.keygen, and set COLLAB_KEY to that same key in every terminal. Full platform-specific setup is in README.md.

```
# Terminal 1
python -m collab.server --tcp-port 9000 --udp-port 9001
# Terminal 2
python -m collab.webui --name alice --web-port 8080
# Terminal 3
python -m collab.webui --name bob --web-port 8081
```

Open http://127.0.0.1:8080 and :8081. Demonstrate online users, private/broadcast chat, file progress and a verified saved file; then stop a client and server to show cleanup. For LAN use, bind the server to 0.0.0.0 and pass its LAN IP using client --host; allow server TCP 9000 and UDP 9001. HTTP remains local. Existing CLI: python -m collab.client --name carol.

Regenerate this report

```
# ReportLab is a report-generation dependency only
python -m pip install reportlab
python -m unittest discover -s tests -v > docs/test_results.txt 2>&1
python docs/build_report.py
```

The generator reads current protocol constants and the completed test log; it refuses an unsuccessful run or a mismatched test inventory. --test-log and --output accept alternate paths. Default output: docs/TECHNICAL_REPORT.pdf. No application sockets or test workers are started by the generator.

Transport interpretation: the brief describes UDP streaming/file reassembly, a TCP handshake, and a techniques section mentioning TCP reliable transfer. This implementation uses encrypted TCP control plus encrypted UDP application data. The report records that design choice and the unimplemented congestion algorithms explicitly.